

Engineering Playbook

EXODUS SYSTEMS INC.

Trainee Engineering Program — Cyberpunk Córdoba 2077

"El código que escribís hoy define la ciudad de mañana."

Índice

1. [Bienvenida](#)
 2. [El Producto](#)
 3. [Arquitectura del Sistema](#)
 4. [Roadmap de Releases](#)
 5. [Modelo de Trabajo](#)
 6. [Estructura del Equipo](#)
 7. [Flujo de Ingeniería](#)
 8. [Estándares de Código](#)
 9. [Pull Requests](#)
 10. [Quality Gates](#)
 11. [Cultura de Ingeniería](#)
-

Bienvenida

Bienvenido/a al **Programa de Ingeniería Trainee de Exodus Systems**.

Durante esta cursada vas a trabajar como parte de una organización de software real, desarrollando el **Cyberpunk Córdoba 2077 — Simulation Core**.

Este entorno replica cómo operan los equipos de ingeniería en la industria. No es solo código — es proceso, disciplina y trabajo en equipo.

Cada concepto que veas en la teoría va a estar **conectado a una feature del sistema**. Algoritmos, estructuras de datos, gestión de memoria — todo aparece acá, en código que corre.

 **Aviso corporativo:** Los memory leaks te bajan el rating de ciudadano. Los warnings sin resolver activan la alerta corporativa.

El Producto

Cyberpunk Córdoba 2077 es un motor de simulación modular ambientado en una versión distópica de Córdoba en el año 2077.

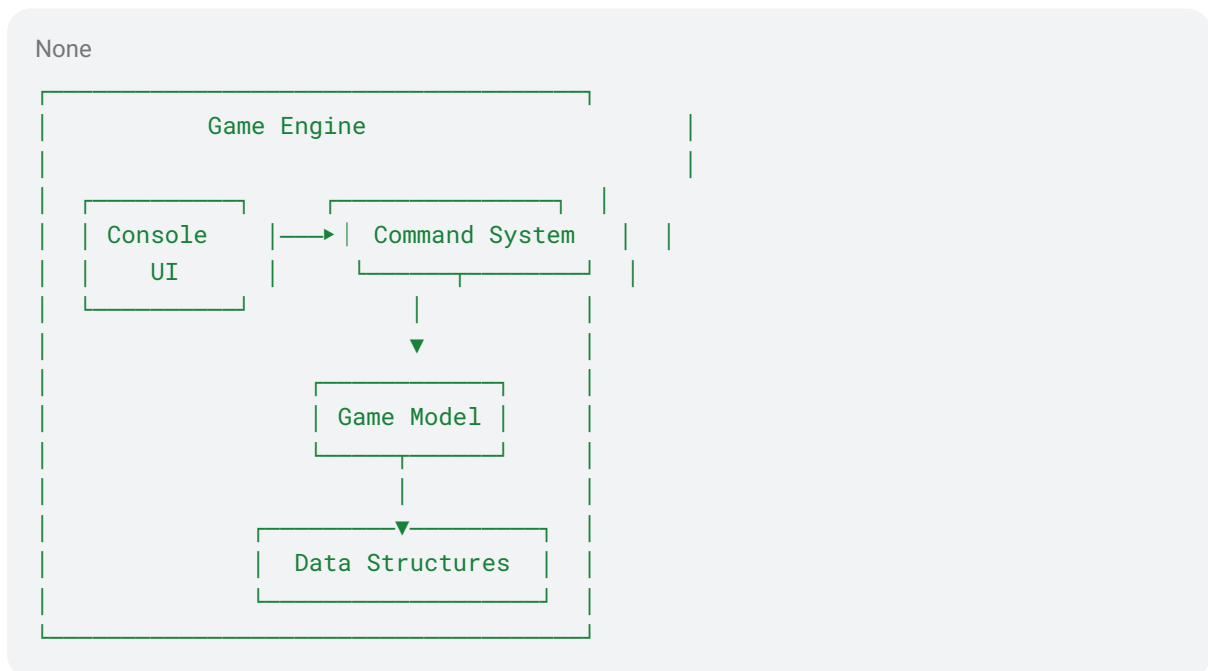
El sistema modela una ciudad futurista con:

- **Entidades** — personajes, items, ubicaciones
- **Estado de simulación** — el mundo cambia con cada acción
- **Interacción del jugador** — comandos que modifican el estado
- **Operaciones algorítmicas** — búsqueda, ordenamiento, traversal

El motor es el **vehículo de aprendizaje**. Cada release introduce nuevos conceptos del curso implementados como features reales del juego.

Arquitectura del Sistema

El sistema sigue una **arquitectura modular**. Cada componente tiene una responsabilidad clara y puede ser desarrollado de forma independiente.



Componentes principales

Console UI Interfaz de texto. Lee el input del usuario, despacha comandos, muestra el output. No contiene lógica de negocio.

Command System Cada comando es un módulo independiente que hereda de la interfaz **Command**. Esto permite que 18 equipos trabajen en paralelo sin pisarse.

Game Model El estado completo de la simulación: jugador, inventario, créditos, ubicación, historial. Es la única fuente de verdad del sistema.

Data Structures A partir de la v0.2.0, las estructuras de la biblioteca estándar se reemplazan progresivamente por implementaciones propias. Aquí es donde la teoría se vuelve código.

Principios de diseño

- **Separación de responsabilidades** — cada módulo hace una sola cosa
- **Componentes testeables** — toda la lógica puede verificarse con unit tests
- **Sin variables globales** — el estado vive en `GameModel`, no en el aire

Roadmap de Releases

El producto evoluciona durante la cursada. Cada release es un **hito de aprendizaje**.

```
None
v0.1.0
Console MVP
|
Comandos
```

Modelo de Trabajo

Trabajás en **equipos de 4 ingenieros/as**.

Cada clase simula un **sprint de producto**, donde los equipos implementan features y entregan Pull Requests.

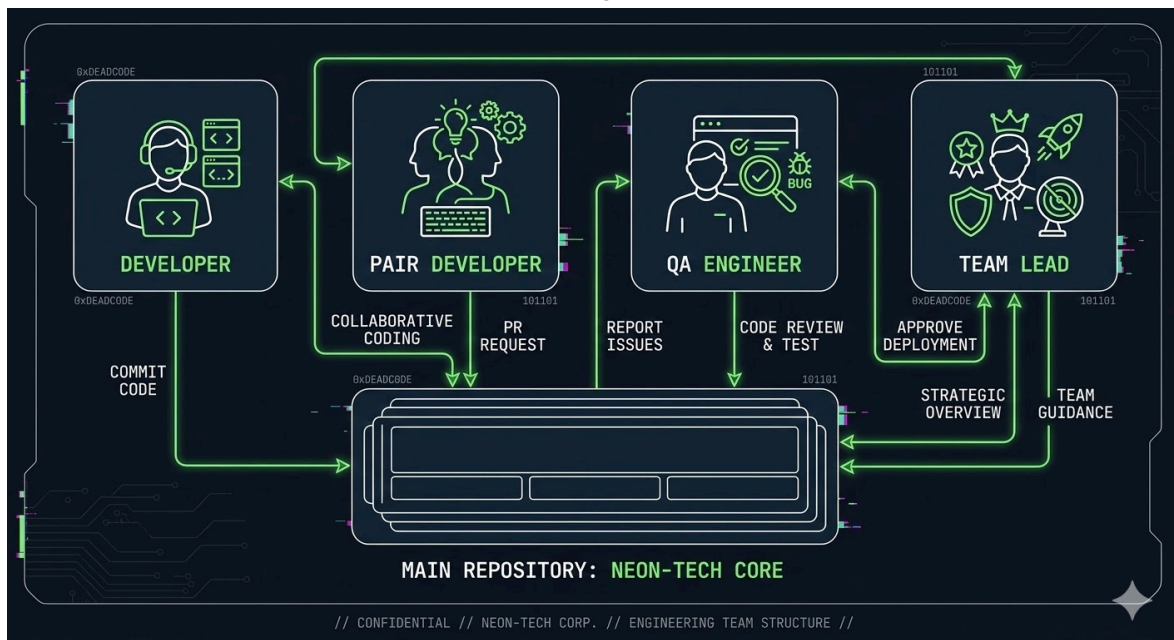
```
None
Clase      = 1 Sprint
Sprint     = 1 Feature por equipo
Feature    = 1 Pull Request
PR merged  = contribución real al producto
```

Los roles **rotan cada clase**. Al cabo de 4 sprints, todos habrán pasado por cada posición.

Estructura del Equipo

 Rol	 Responsabilidad
Dev Principal	Implementa la feature
Pair Programmer	Codea junto al Dev Principal — pair programming activo, no espectador
Tester	Escribe los tests, valida el comportamiento, define casos borde
Team Lead	Arma el PR, escribe la descripción, hace el checklist, es responsable de que el trabajo sea de calidad.

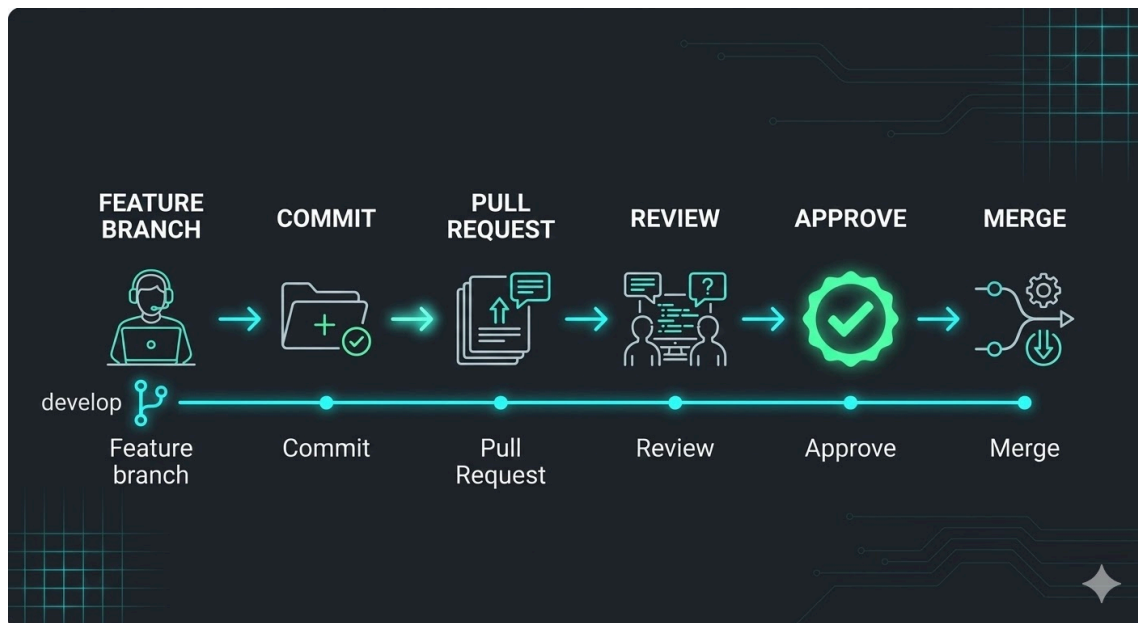
💡 El **Team Lead** no es el más experimentado — es el responsable de esa clase. El rol entrena criterio, no solo código.



Flujo de Ingeniería

```

None
development
  ↳ feature/nombre-significativo    ← creás tu branch acá
    ↳ commits                        ← trabajás acá
      ↳ Pull Request                 ← pedís review acá
        ↳ merge                      ← se integra al producto
          ↳ development actualizado → próximo sprint
  
```



Paso a paso

1. Crear el branch

```
Shell
git checkout development
git pull origin development
git checkout -b feature/nombre-significativo
```

Convención obligatoria: [feature/help-command](#), [feature/status-command](#), etc.

2. Implementar

Seguí la arquitectura. Si tenés dudas sobre cómo estructurar un comando, revisá los ejemplos en [src/commands/](#).

3. Tests y formato

```
Shell
cd build && cmake .. && make && ctest --verbose
clang-format -i src/./A.cpp src/./B.h
```

4. Commit

```
Shell
git add .
git commit -m "feat: [Descripción breve de lo nuevo]"
```

5. Push y PR

```
Shell
git push origin feature/nombre-significativo
```

Abrís el PR desde GitHub con el template. El Tech Lead de turno es el responsable de que esté completo.



Estándares de Código

Reglas obligatorias

✓ Hacer	✗ No hacer
Una clase por archivo	Lógica de negocio en <code>Console.cpp</code>
Heredar de <code>Command</code> para todo comando	Variables globales
Estado del juego solo en <code>GameModel</code>	Hardcodear valores que deberían venir del modelo
<code>nullptr</code> en lugar de <code>NULL</code>	Código comentado sin explicación
Nombres descriptivos	Variables de una letra (excepto índices)

La interfaz base — no modificar

```
C/C++
// Command.h - READ ONLY
class Command {
public:
    virtual void execute(GameModel& model) = 0;
    virtual std::string name() const = 0;
    virtual std::string description() const = 0;
    virtual ~Command() = default;
};
```

Tu comando implementa estos tres métodos. Nada más.

Complejidad

Aunque la implementación sea trivial, el PR debe mencionar la complejidad del algoritmo principal. Es un hábito que vale adquirir desde el primer sprint.

Pull Requests

Cada PR **usa el template** de `.github/pull_request_template.md`.

```
None
## 🎯 Feature
¿Qué implementa este PR?

## 🏗️ Diseño
¿Por qué elegiste este approach?

## 📊 Complejidad
Big-O del algoritmo principal

## 🧪 Tests
Lista de tests agregados

## 📸 Evidencia
Output de ejemplo del comando funcionando
```

Convención de títulos

```
None
feat: add help command
feat: add inventory command
fix: handle empty state in status command
```

Quality Gates

Un PR **no se mergea** si falla alguno de estos gates:

```
None
[ ] Compila sin warnings
[ ] Todos los tests pasan
[ ] clang-format aplicado
[ ] PR description completa con template
[ ] Mínimo 2 tests: caso normal + caso borde
[ ] Sin memory leaks
[ ] No toca archivos de otros equipos
```


● Un gate en rojo bloquea el PR. Sin excepciones.

PR of the Week

Al inicio de cada clase se anuncia el **PR of the Week** del sprint anterior, proyectado en pantalla.

Criterio	Qué se evalúa
 Diseño	¿La solución es clara y bien pensada?
 Tests	¿Cubren casos reales, no solo el happy path?
 Documentación	¿La PR description explica las decisiones?
 Calidad	¿El código está limpio, sin warnings, sin magic numbers?

El equipo ganador explica su PR en 3 minutos. No el código — las **decisiones**.

Cultura de Ingeniería

En Exodus Systems valoramos:

- **Claridad técnica** — el código se escribe para ser leído por otras personas
- **Disciplina de ingeniería** — los estándares existen por razones, no por burocracia
- **Desarrollo colaborativo** — el pair programming no es un rol pasivo
- **Mejora continua** — cada PR es una oportunidad de aprender algo nuevo

El objetivo final del producto es un **motor de simulación gráfico** con pathfinding sobre el mapa de Neo-Córdoba. Cada línea de código que mergeás hoy es un bloque de ese sistema.

Tu objetivo no es solo escribir código que funcione. Es desarrollar **criterio de ingeniería**.

Eso es lo que distingue a un programador de un ingeniero de software.

Planilla de equipos



Protocolo de Gestión de Sprints: Estructura de Equipos Multi-Core

1. Resumen Ejecutivo

Esta [planilla de cálculo](#) constituye el **Single Source of Truth (SSoT)** para la distribución de carga operativa y responsabilidad técnica del proyecto. Cada pestaña representa una unidad funcional de ingeniería (**Equipos**) y garantiza la rotación equitativa de responsabilidades para evitar silos de conocimiento.

2. Gobernanza de Roles y Rotación Automática

Para asegurar que cada ingeniero desarrolle competencias tanto en implementación como en aseguramiento de calidad y liderazgo, se ha implementado un **Kernel de Asignación Dinámica**.

Fórmula de Implementación (Columna C):

None

```
=ELEGIR(RESIDUO(ENTERO((HOY()-FECHANUM("2026-03-02"))/7) + FILA(A1); 4) + 1;  
"Dev Principal"; "Pair Programmer"; "Tester"; "Team Lead")
```

Directiva Operativa: La rotación se ejecuta de forma determinista cada lunes a las 00:00 UTC. No se permiten cambios manuales en la asignación de roles sin aprobación de la dirección técnica.

3. Matriz de Responsabilidades (RACI)

Rol	Entregable Clave	Estándar de Desempeño
Dev Principal	Código de Producción	Implementación de features alineada a los specs.
Pair Programmer	Revisión en Tiempo Real	Soporte táctico y prevención de deuda técnica inmediata.
Tester	Suite de Pruebas	Cobertura de tests unitarios y validación de casos borde.
Team Lead	Asegurar la calidad de PR	Verificación final de calidad y cumplimiento del Definition of Done (DoD).

4. Registro de Actividad y Accountability

Es obligatorio que cada miembro del equipo documente su impacto semanal. Al finalizar el sprint, el usuario debe realizar un **Log de Tareas**:

1. Situarse sobre la celda de su nombre.
2. Insertar un **Comentario (Ctrl+Alt+M)**.
3. Registrar el resumen de ejecución siguiendo el formato:
 - [ROL]: [BREVE DESCRIPCIÓN DE LOGROS]
 - [IMPEDIMENTOS]: [BLOQUEOS ENCONTRADOS]

How to tareas

Guía EXODUS: Cómo tomar una Issue, trabajar en equipo y entregar con calidad

Esta guía es **operativa**: te dice qué hacer y en qué orden. No buscamos definiciones largas: buscamos **flujo**, **claridad** y **calidad**.

Política de asignación de tareas (FCFS) La asignación de Issues se rige por el criterio **FCFS (First Come, First Served)**: el **primer equipo** que se **asigna formalmente** una tarea en GitHub (assignee + comentario de inicio) obtiene la **prioridad y ownership** de esa Issue.

Para evitar duplicación de esfuerzo y conflictos de integración:

- **No se permite** que dos equipos trabajen la **misma** tarea en paralelo.
- Si una Issue ya está asignada o en progreso, el equipo debe seleccionar **otra** disponible.
- Ante cualquier duda sobre disponibilidad o solapamiento, escalen al **Project Lead** antes de comenzar.

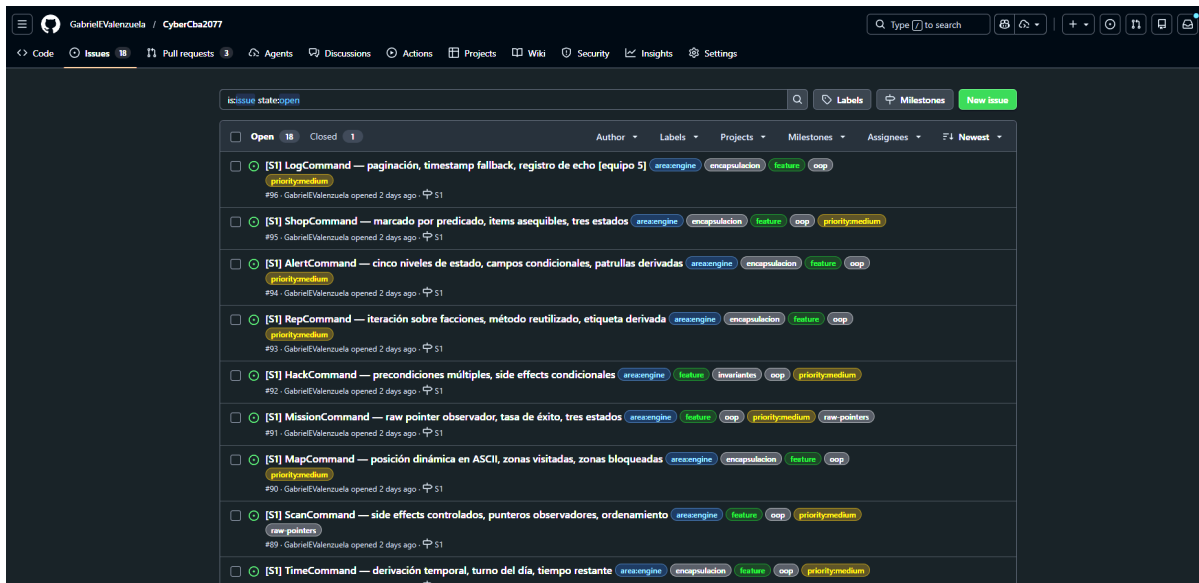
✓ Objetivo: maximizar throughput del sprint manteniendo trazabilidad y evitando retrabajo.

1) Qué es una Issue (en EXODUS)

Una **Issue** es nuestra **tarea de sprint**: la unidad mínima de trabajo que se puede:

- asignar a alguien
- ejecutar
- revisar
- validar
- y dar por “terminada” con criterios claros

Pensala como un **contrato de entrega**: *qué se espera y cómo se valida*.

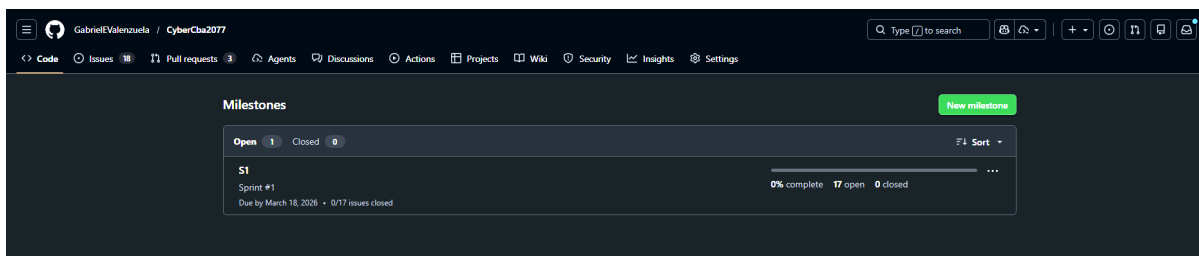


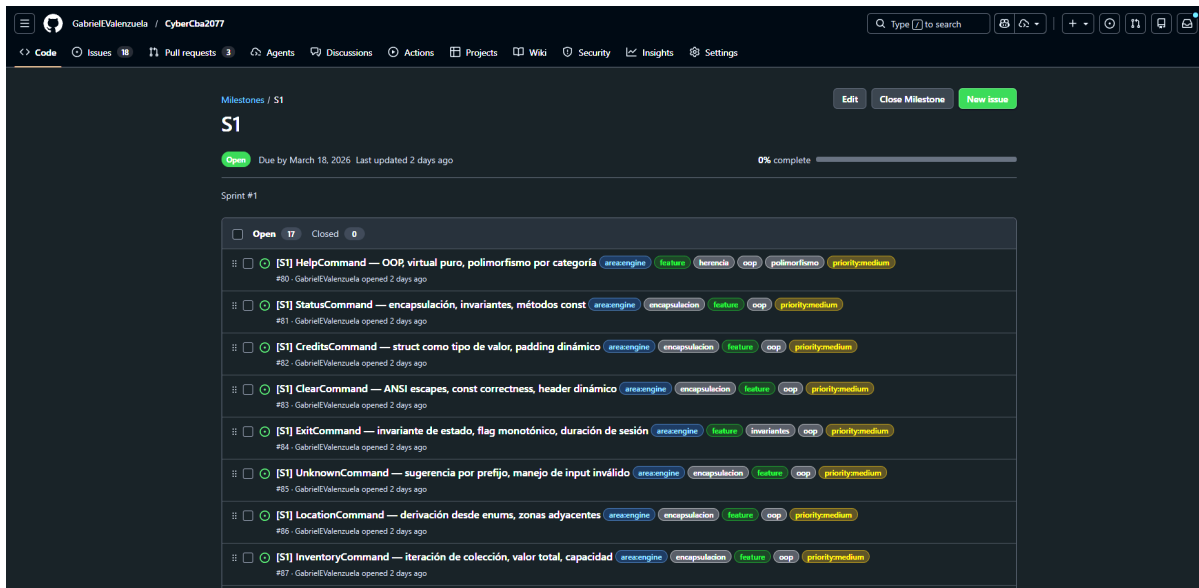
2) Qué es un Sprint (en EXODUS)

Un **Sprint** es una ventana corta de tiempo (ej.: 1–2 semanas) donde:

- elegimos un conjunto de issues
- las movemos de “To Do” → “In Progress” → “Done”
- y dejamos el proyecto **más funcional y estable** al final

No es “hacer por hacer”: es **entregar valor verificable** con disciplina.



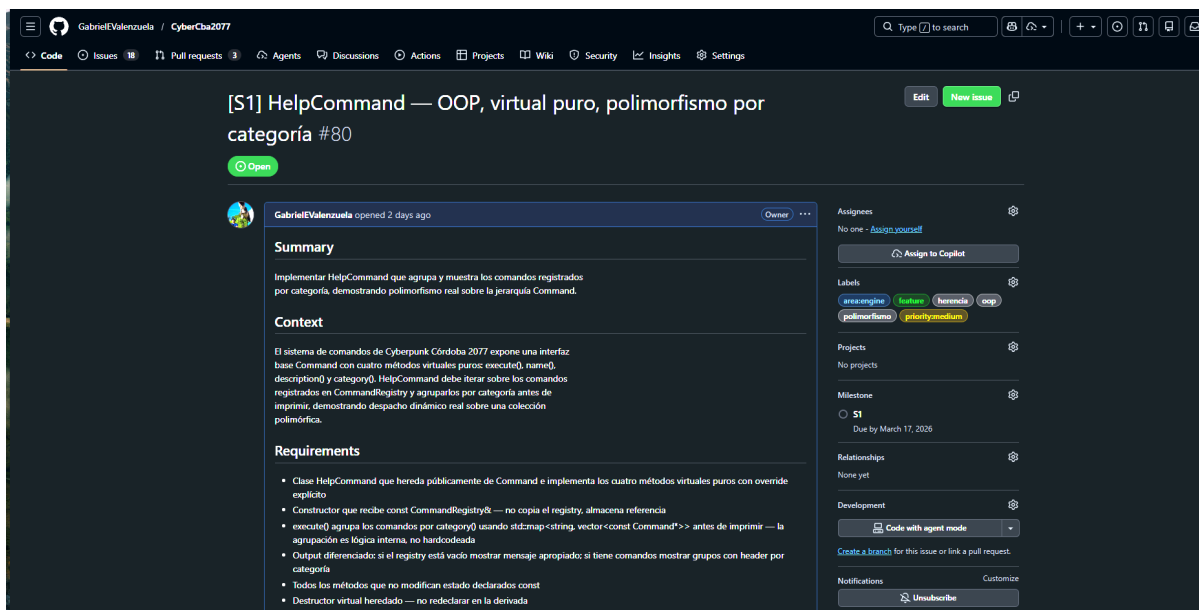


3) Paso a paso: cómo tomar una tarea (Issue) como se hace en industria

Paso 1 — Léé la Issue como si fueras el reviewer

Antes de codear, asegurate de entender:

- objetivo (qué cambia)
- criterios de aceptación (qué significa “listo”)
- alcance (qué *entra* y qué *no entra*)



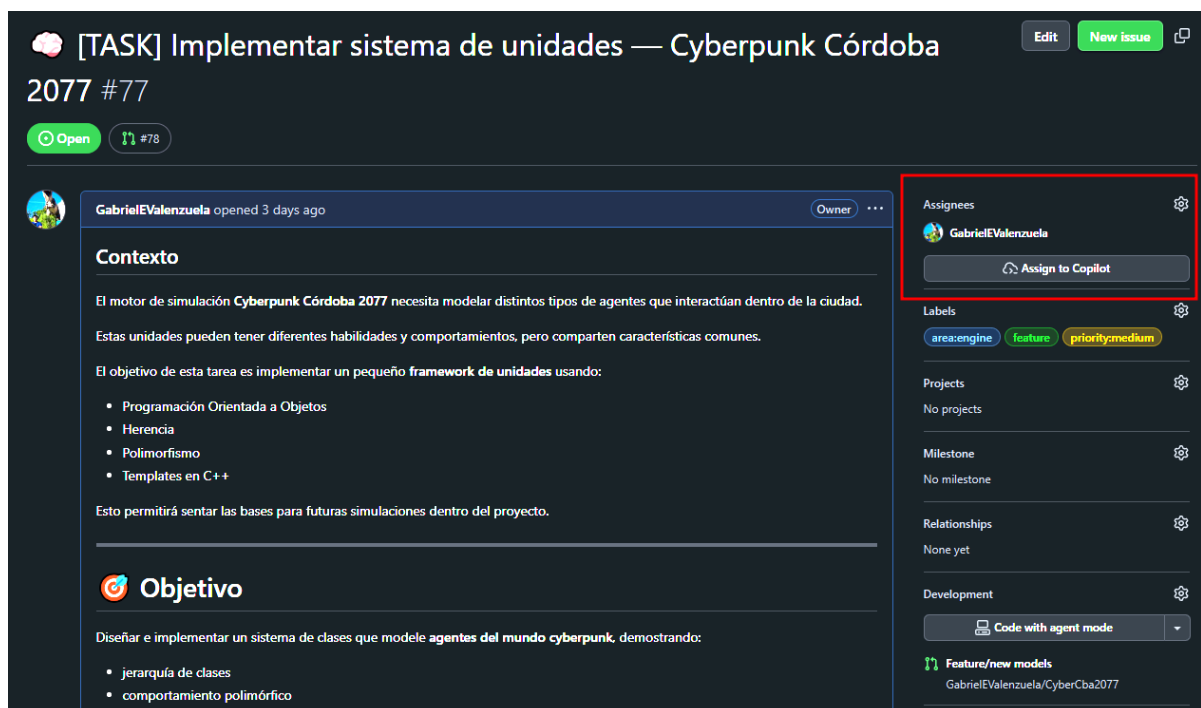
✓ Regla EXODUS: **si algo no está claro, se pregunta antes de codear**. Eso es velocidad real (menos retrabajo).

Paso 2 — Asignate la Issue

Toda Issue debe tener dueño para evitar trabajo duplicado y para que el equipo sepa quién está ejecutando.

En GitHub:

- **Assignees** → **asignarte a vos mismo y a tu equipo**



✓ Resultado esperado: la Issue queda con el nombre de todo tu equipo

Paso 3 — Dejás un comentario de “arranco”

Comentario corto, estilo profesional:

- “Tomo la tarea. Voy por enfoque X. Primer PR hoy/mañana. Aviso si hay bloqueos.”

✓ Esto alinea expectativas y crea trazabilidad.

Paso 4 — Definí roles en 2 minutos (sin burocracia)

Antes de escribir mucho código, el equipo define quién hace qué:

- **Dev Principal:** implementa la feature
- **Pair Programmer:** programa activamente junto al Dev Principal
- **Tester:** define casos borde y valida comportamiento (tests + manual)
- **Team Lead:** arma PR, descripción, checklist y asegura calidad

💡 Importante: el **Team Lead no es “el más experimentado”**. Es el responsable de **esa entrega**: entrena criterio, comunicación y ownership.

4) Consejos para trabajar en equipo en una Issue (lo que hace que salga bien)

4.1 “Plan de 5 líneas” antes de codear

No un documento largo. Solo 5 líneas para evitar caos:

1. Qué vamos a cambiar
2. Dónde va a vivir (módulos/archivos)
3. Cómo lo vamos a probar
4. Casos borde mínimos
5. Qué NO vamos a hacer (control de scope)

✅ Resultado: PRs más chicas y decisiones más claras.

4.2 Pair programming real (no espectador)

El Pair Programmer:

- propone alternativas
- revisa en tiempo real
- detecta bugs temprano
- discute decisiones de diseño con el Dev Principal

✅ Resultado: menos bugs y más aprendizaje compartido.

4.3 Tester entra temprano (no al final)

El Tester no “aparece cuando ya está todo hecho”. Entra temprano para:

- definir qué significa “funciona”
- proponer casos borde
- anticipar fallas típicas
- orientar tests (GTest) y validación manual

✅ Resultado: menos retrabajo, más calidad.

4.4 PR chica, legible y con evidencia

Lo que hace que una PR se apruebe rápido:

- alcance acotado
- descripción clara (TL;DR)
- checklist completo
- evidencia (tests, logs, screenshot, output)

✅ Resultado: revisión más rápida, sprint más fluido.

5) Mentalidad EXODUS (cómo ganamos velocidad con calidad)

- Si algo es confuso → **preguntá temprano**
- Si se rompe → **se reporta y se arregla**, no se tapa
- Si la PR es enorme → **se parte**
- Si falta test → **se agrega mínimo viable**
- Si hay duda de diseño → **se deja explicitada en el PR**

El objetivo no es “hacer muchas issues”. El objetivo es **entregar valor estable y mantenible**.

Si seguimos este flujo, el proyecto avanza más rápido y mejor. 🚀

ADRs y RF/RnF

Cyberpunk Córdoba 2077 — Simulation Core

Architecture Decision Records + Requirements Specification

Exodus Systems Inc. | Trainee Engineering Program Document Version: 2.0

Índice

- [ADRs](#)
 - [Requerimientos Funcionales](#)
 - [Requerimientos No Funcionales](#)
-

ADRs

Los ADRs documentan **por qué** se tomaron las decisiones de arquitectura, no solo qué se decidió. Son el historial técnico del producto.

ADR-001 — Arquitectura modular por capas

Estado:  Aceptado

Contexto

El producto es un RPG CLI desarrollado por 18 equipos en paralelo sobre un mismo repositorio. Sin separación explícita de responsabilidades, los conflictos de merge y el acoplamiento entre módulos destruirían la cadencia de desarrollo.

Decisión

Adoptar una arquitectura en capas con ownership claro:

```
None
include/ + src/
  core/      → estado del juego (GameModel, PlayerState, WorldState, Types)
  commands/  → lógica de cada acción del jugador (Command Pattern)
  ui/        → entrada/salida por consola (y luego raylib)
  ds/        → estructuras de datos propias
  tests/     → pruebas automatizadas por módulo
```

benchmarks/ → medición de performance de estructuras propias

Consecuencias

- Cada equipo tiene ownership de su archivo — merge conflicts casi imposibles
- Interfaces explícitas entre capas — testeabilidad alta
- La capa `ui/` es reemplazable por raylib sin tocar el modelo
- Los alumnos aprenden separación de responsabilidades haciendo, no leyendo

ADR-002 — MVP inicial con interfaz CLI

Estado: ☒ Aceptado

Contexto

El objetivo final es un RPG con interfaz gráfica (raylib). Pero introducir gráficos en v0.1.0 ocultaría problemas de diseño, dificultaría el testing automatizado y consumiría tiempo de clase en setup de bibliotecas gráficas.

Decisión

Construir primero un **RPG jugable por consola** que valide mecánicas del juego, arquitectura y estructuras de datos. La UI gráfica se monta sobre un modelo ya probado.

None

v0.1.0	→ CLI RPG funcional	(comandos, estado, mecánicas básicas)
v0.2.0	→ CLI RPG + DS propias	(estructuras reemplazando STL)
v0.3.0	→ CLI RPG completo	(misiones, combate, entidades)
v1.0.0	→ RPG gráfico con raylib	(render 2D, mapa, pathfinding)

Consecuencias

- Tests automatizados desde el día 1 — sin fricción de render
- Los alumnos entienden qué es un "modelo" antes de ver una pantalla gráfica
- La migración a raylib en vX.0.0 es un refactor de presentación, no de lógica

ADR-003 — Command Pattern para acciones del jugador

Estado: ☒ Aceptado

Contexto

En un RPG CLI, el jugador interactúa escribiendo comandos (`attack`, `inventory`, `move`, `status`). Si toda la lógica vive en `Console.cpp`, 18 equipos pisarían el mismo archivo constantemente.

Decisión

Cada acción del jugador es un comando independiente que hereda de la interfaz `Command`:

```
C/C++
class Command {
public:
    virtual void execute(GameModel& model) = 0;
    virtual std::string name() const = 0;
    virtual std::string description() const = 0;
    virtual ~Command() = default;
};
```

Los comandos se registran en `CommandRegistry`. `Console.cpp` solo lee input y despacha — no contiene lógica de juego.

Consecuencias

- Cada equipo tiene su propio `.cpp` — cero conflictos estructurales
- Agregar un comando nuevo es agregar un archivo, no modificar uno existente
- Los comandos son testeables de forma aislada
- Escala naturalmente: en v1.0.0 raylib dispara los mismos comandos

ADR-004 — GameModel como única fuente de verdad

Estado:  Aceptado

Contexto

En un RPG, el estado del mundo es complejo: jugador, inventario, misiones activas, entidades cercanas, historial de acciones, nivel de alerta. Sin un modelo centralizado, cada comando accedería a variables dispersas y el estado se volvería inconsistente.

Decisión

`GameModel` es el único objeto que contiene el estado completo de la simulación. Todo comando recibe una referencia a `GameModel` y opera sobre él.

```
C/C++  
// Todos los comandos reciben esto - nada más  
void execute(GameModel& model) override;
```

Para v0.2.0 en adelante, `GameModel` se partirá en sub-estados con ownership claro:

```
None  
GameModel  
├─ PlayerState   → HP, créditos, nivel, reputación  
├─ WorldState    → ubicación actual, zonas, hora del sistema  
├─ Inventory     → CustomVector<Item> (v0.2.0)  
├─ MissionLog    → LinkedList<Mission> (v0.2.0)  
├─ EventQueue    → Queue<GameEvent> (v0.2.0)  
└─ ActionHistory → Stack<Action> (v0.2.0 - para undo)
```

Consecuencias

- Estado predecible y testeable
- Sin variables globales en ningún comando
- La partición en v0.2.0 es natural — los alumnos ya conocen el modelo para entonces

ADR-005 — Estructuras de datos propias integradas al dominio del juego

Estado: ☒ Aceptado

Contexto

El objetivo del curso es que los alumnos implementen y comprendan estructuras de datos. Si las estructuras propias son ejercicios desconectados del producto, el aprendizaje es superficial. Si reemplazan estructuras reales del RPG, el aprendizaje tiene consecuencias medibles.

Decisión

Cada estructura propia reemplaza una estructura STL en un componente real del juego:

Estructura propia	Reemplaza	Componente del RPG
<code>CustomVector<T></code>	<code>std::vector</code>	Inventario del jugador
<code>LinkedList<T></code>	<code>std::list</code>	Log de misiones completadas

Estructura propia	Reemplaza	Componente del RPG
<code>DoublyLinkedList<T></code>	<code>std::deque</code>	Historial de comandos navegable
<code>Stack<T></code>	<code>std::stack</code>	Sistema de undo de acciones
<code>Queue<T></code>	<code>std::queue</code>	Cola de eventos del mundo

La STL puede usarse en contextos no pedagógicos (strings, parsing de input).

Consecuencias

- El alumno ve el impacto real de su implementación en el juego
- Los benchmarks miden su estructura contra `std::` — hay un número concreto que defender
- Si la implementación tiene un bug, el juego falla de forma visible

ADR-006 — GitHub Flow con PRs obligatorios y CODEOWNERS

Estado:  Aceptado

Contexto

Con 18 equipos, la disciplina de integración no puede depender de buena voluntad. Necesita enforcement estructural.

Decisión

- Todo cambio entra por branch + Pull Request + CI verde
- `CODEOWNERS` protege los archivos de arquitectura base:

```
None
# .github/CODEOWNERS
include/core/           @instructor
include/commands/Command.h @instructor
src/ui/Console.cpp      @instructor
CMakeLists.txt          @instructor
.github/                 @instructor
```

- `CommandRegistry.cpp` tiene una sección delimitada donde cada equipo agrega su línea:

C/C++

```
// === TEAM COMMANDS - cada equipo agrega su línea acá ===  
registry.add(std::make_unique<HelpCommand>());  
registry.add(std::make_unique<StatusCommand>());  
// === END TEAM COMMANDS ===
```

Consecuencias

- Los archivos críticos no pueden mergearse sin aprobación del instructor
 - Los alumnos aprenden el flujo real de contribución a un repo compartido
 - `CommandRegistry` permanece estable aunque 18 equipos lo toquen
-

ADR-007 — CI como quality gate con `-Werror`

Estado:  Aceptado

Contexto

El quality gate "compila sin warnings" solo tiene valor si algo lo enforcea automáticamente. Sin CI, es una promesa que se rompe bajo presión de deadline.

Decisión

El pipeline CI corre en todo PR y verifica:

1. Compilación con `-Wall -Wextra -Werror`
2. Tests (GTest — todos deben pasar)
3. Formateo (`clang-format --dry-run --Werror`)
4. Análisis estático básico (`cppcheck`)

Un PR con CI rojo no puede mergearse. Sin excepciones.

Consecuencias

- Los alumnos aprenden que un warning es un error desde el día 1
 - El instructor no necesita revisar manualmente compilación ni formato
 - `-Werror` duele la primera semana y forma hábito para siempre
-

ADR-008 — Benchmarks desde v0.1.0

Estado:  Aceptado

Contexto

En v0.2.0 los alumnos van a reemplazar `std::vector` por su `CustomVector`. Sin medición de base, no hay forma de saber si su implementación es mejor, igual o peor.

Decisión

Los benchmarks (Google Benchmark) se incluyen en el scaffolding desde v0.1.0, corriendo sobre estructuras STL. En v0.2.0 el mismo benchmark corre sobre las estructuras propias y el alumno ve la diferencia.

Consecuencias

- La complejidad algorítmica deja de ser teórica — hay nanosegundos que defender
 - El alumno puede optimizar su implementación con feedback inmediato
 - La infraestructura de benchmark no es una sorpresa en v0.2.0
-

ADR-009 — Raylib solo como capa de presentación

Estado:  Propuesto — target v1.0.0

Contexto

La futura UI gráfica no debe contaminar la lógica del RPG. Si raylib entra en `GameModel` o en los comandos, el testing se vuelve imposible y la arquitectura se degrada.

Decisión

Raylib se incorpora en v1.0.0 como capa de visualización que consume `GameModel` y dispara `Commands`. No contiene lógica de juego.

None

RaylibUI → lee `GameModel` → renderiza estado

RaylibUI → captura input → dispara `Command` → modifica `GameModel`

El modelo y los comandos no saben que raylib existe.

Consecuencias

- La migración de CLI a gráfico es un refactor de `Console.cpp`, no del modelo
 - Los tests del modelo y los comandos siguen siendo válidos en v1.0.0
 - Los alumnos ven en práctica qué significa "separación entre lógica y presentación"
-

Requerimientos Funcionales

Organizados por release. Cada RF tiene un owner claro en la arquitectura.

v0.1.0 — CLI RPG MVP

RF-001 El sistema debe iniciar una sesión de juego con estado inicial válido. **Owner:** `GameModel`

RF-002 El sistema debe ofrecer un loop de interacción por consola que lea comandos del jugador. **Owner:** `Console`

RF-003 El sistema debe reconocer y ejecutar comandos textuales del jugador. **Owner:** `CommandRegistry + Command`

Comandos mínimos del Sprint S1:

```
None
help · status · credits · exit · clear
location · inventory · time · scan · map
mission · hack · rep · alert · shop · log · echo
```

RF-004 El sistema debe manejar comandos inválidos con un mensaje claro sin interrumpir la ejecución. **Owner:** `UnknownCommand`

RF-005 El sistema debe exponer el estado del jugador: nombre, créditos, ubicación, nivel de alerta. **Owner:** `StatusCommand + GameModel`

RF-006 El sistema debe mostrar el inventario del jugador con capacidad actual y máxima. **Owner:** `InventoryCommand + GameModel`

RF-007 El sistema debe representar la ubicación del jugador en el mapa de Neo-Córdoba. **Owner:** `LocationCommand + MapCommand + GameModel`

RF-008 El sistema debe registrar un historial de acciones del jugador. **Owner:** `LogCommand + GameModel`

RF-009 El sistema debe representar la reputación del jugador con distintas facciones. **Owner:** `RepCommand + GameModel`

RF-010 El sistema debe poder cerrarse de manera segura, liberando todos los recursos. **Owner:** `ExitCommand`

v0.2.0 — Data Structures Integration

RF-011 El inventario debe estar implementado sobre `CustomVector<Item>` propio.

`Owner: ds/CustomVector + GameModel::Inventory`

RF-012 El historial de misiones debe estar implementado sobre `LinkedList<Mission>` propia. `Owner: ds/LinkedList + GameModel::MissionLog`

RF-013 El sistema debe soportar undo de la última acción mediante una `Stack<Action>` propia. `Owner: ds/Stack + GameModel::ActionHistory`

RF-014 El sistema debe administrar una cola de eventos del mundo mediante `Queue<GameEvent>` propia. `Owner: ds/Queue + GameModel::EventQueue`

RF-015 El sistema debe poder navegar el historial de comandos hacia adelante y atrás con `DoublyLinkedList<Command>`. `Owner: ds/DoublyLinkedList + Console`

v0.3.0 — Simulation Logic

RF-016 El sistema debe soportar combate básico entre el jugador y entidades hostiles.

`Owner: GameModel + CombatCommand`

RF-017 El sistema debe gestionar un sistema de misiones con estados: pendiente, activa, completada, fallida. `Owner: MissionCommand + GameModel::MissionLog`

RF-018 El sistema debe aplicar reglas de negocio del RPG: daño, curación, consumo de créditos, equipamiento. `Owner: GameModel`

RF-019 El sistema debe soportar búsqueda de ítems en inventario con complejidad explícita documentada. `Owner: InventoryCommand + CustomVector`

RF-020 El sistema debe modelar entidades del mundo: NPCs, enemigos, comerciantes, con estado propio. `Owner: WorldState + ScanCommand`

v1.0.0 — Visual Interface

RF-021 El sistema debe renderizar el mapa de Neo-Córdoba como un grafo visual con raylib. `Owner: RaylibUI`

RF-022 El sistema debe implementar pathfinding entre zonas del mapa (Dijkstra o BFS).
Owner: `WorldState` + grafo de zonas

RF-023 El sistema debe visualizar el estado del jugador (HP, créditos, alerta) en un HUD.
Owner: `RaylibUI` + `GameModel`

RF-024 La lógica del juego debe ser completamente independiente de raylib. Owner: `Arquitectura` – verificable por tests

Requerimientos No Funcionales

Arquitectura y diseño

RNF-001 — Modularidad Cada módulo tiene una responsabilidad única. Agregar un comando nuevo no requiere modificar módulos existentes.

RNF-002 — Separación entre lógica y presentación `GameModel` y los comandos no tienen dependencias de `Console` ni de raylib. La UI es reemplazable.

RNF-003 — Sin variables globales El estado del juego vive exclusivamente en `GameModel`. Los comandos reciben el modelo por referencia.

RNF-004 — Ownership explícito de memoria Cada objeto tiene un dueño claro. Se prefiere RAII y smart pointers. Sin raw owning pointers en código nuevo.

Calidad de código

RNF-005 — Compilación sin warnings El sistema compila con `-Wall -Wextra -Werror`. Un warning es un error.

RNF-006 — Sin memory leaks El sistema pasa `valgrind --leak-check=full` en su flujo principal sin errores.

RNF-007 — Consistencia de estilo Todo el código respeta la configuración de `clang-format` del repositorio. El CI verifica esto automáticamente.

RNF-008 — Legibilidad Los nombres de clases, métodos y variables son descriptivos. El código no requiere comentarios para entenderse.

Testing y CI

RNF-009 — Testabilidad Todo componente con lógica de negocio puede testearse de forma aislada sin levantar la UI.

RNF-010 — Cobertura mínima por feature Cada PR incluye mínimo 2 tests: caso normal y caso borde. El CI verifica que los tests existan y pasen.

RNF-011 — CI como gate obligatorio Ningún PR se mergea con CI rojo. El pipeline cubre: compilación, tests, formateo y análisis estático.

RNF-012 — Benchmarks como evidencia En v0.2.0, cada estructura propia incluye un benchmark que compara su performance contra la equivalente STL.

Proceso y colaboración

RNF-013 — Trazabilidad completa Todo cambio entra por branch + PR + review. No hay commits directos a `main`.

RNF-014 — Documentación en el PR Cada PR explica qué implementa, por qué se eligió ese approach, la complejidad del algoritmo principal y evidencia de funcionamiento.

RNF-015 — Escalabilidad con 18 equipos La arquitectura permite trabajo paralelo de 18 equipos sin conflictos estructurales. Verificable: cada equipo toca solo sus archivos.

RNF-016 — Releases versionadas y estables Cada release representa un estado compilable, con todos los tests en verde, etiquetado con semver.

Performance

RNF-017 — Complejidad documentada Todo PR menciona la complejidad Big-O del algoritmo principal, aunque sea $O(1)$.

RNF-018 — Estructuras propias respetan complejidades esperadas

`CustomVector::push_back` es $O(1)$ amortizado. `LinkedList::insert` es $O(1)$. Las desviaciones son bugs, no features.

Evolución del producto

RNF-019 — Evolución incremental sin reescritura El núcleo del sistema (GameModel + Command Pattern) se mantiene estable de v0.1.0 a v1.0.0. Las releases agregan capas, no reemplazan el modelo.

RNF-020 — Coherencia pedagógica Las decisiones técnicas exponen los conceptos del curso, no los ocultan. Si un alumno pregunta "¿por qué usamos esto?", la arquitectura tiene una respuesta concreta.